前有数据结构

CoreLoop

UNKNOWN VOICE

是的，没错，这片大陆叫做《数据结构》，是历代先贤们以智慧为剑刃开辟的疆土，也是无数求知者在 0 与 1 的荒原上寻找真理的朝圣之地。寻理者皆向北而行，并了解了古老律法的真相。

This land is called Data Structures. It is a realm carved out by sages of past generations with wisdom as their blade, and it is also a sacred pilgrimage site where countless seekers of knowledge search for truth in the wilderness of 0 and 1. In venturing north, the seekers discover the truth of logic.

位不见王影，然火焰映彻苍穹。当传承火焰燃烧之时，钟声将响彻四周，进而唤醒遍布这片大陆的思渡者们。

时秩双王 — 栈和队列，分界统御者 — 树与二叉树，以及万象之都的孤独王者 — 图。不过啊，王者们一定会筑起不可逾越的高墙吧。而无火的余灰将会纷至踏来，那是无名、成不了薪，被命运牵绊的思渡者。但也正因如此，灰烬才会如此渴求余火吧。

落叶捎来消息：在雾的彼端，我们的故乡“交界地”，那伟大的艾尔登法环已经破碎，“永恒女王”玛莉卡销声匿迹，在黑刀阴谋之夜，“黄金”葛德文最先失去性命，玛莉卡之子 – 诸位半神拿到艾尔登法环的碎片。却因为那股力量堕入歧途、陷入疯狂、引发碎片战争..... 在那场无王存在的战争最后 – 无上意志放逐了他们。奥，所以啊，褪色者啊 – 依旧无法永眠的死者啊，那许久以前我们失去的赐福，正在出声呼唤。“蛮荒地王者”荷莱露。光耀金面具、“死眠少女”菲雅、受尽唾弃的食粪者、“百智爵士”基甸奥夫尼尔。..... 那失去的赐福又再一次，回到默默无名的褪色者身上。朝雾的彼端前进，抵达交界地，觐见艾尔登法环 – 成为艾尔登之王吧。

——— BONFIRE HIT ———

欢迎来到营火，无火的余灰大人。我正是维火女，专门维护营火以及服侍您的人。

正如您所见，穿过山隘，前方就是数据结构大陆中不显眼的一角——《前有数据结构》，这片大陆的守护神亦是开创者，大家称其为 CoreLoop。如果您为了觐见**树、图**等王者们而需要帮助，我很愿意为此帮忙。

首先，CoreLoop 托我带给您以下谏言，愿它们能照亮您前行的路。

1. 《前有数据结构》假设您已经有了比较好的 C 语言基础，但他也会在某些合适的地方强调某些 C 语言的知识点。
2. 在您思渡的过程中，遇到的所有“思维拓展”**都！可！以！不！必！停！留！！**我向您保证，即使跳过这些，您也决不会错过任何核心内容。您可根据适合自己的方式选择是否驻足。

如果你对本书中的内容存在疑问，欢迎联系作者。【联系方式】

许多地方添加一些内容，能够对正文有很好的补充及/或帮助大家理解的作用，但是为了保证在本书中不过分多地引入与《数据结构》本身无关的内容，作者并未给出这些“DLC”。如果想要了解，请参考我的视频。

知识绝不会少，但更重要的是思考。

很多有意思的东西都是藏在习题里面。

即使引导早已破碎，也请您成为数据结构之王。

恭喜你找到了第一个篝火。你可以在此处整理背包，回忆这段旅程的所见所闻，并且将所得卢恩转化为力量，并解锁一些新的能力。后续可以在适合的篝火处和读者说说话。

继续向前，便是 CoreLoop 所开创的大陆。在旅途中，若我的出现偶尔成了您的负累，还请您不必驻足与我的谏言。我并非大陆的探拓者，因此这不会让您错过任何数据结构大陆的美丽风景。无论如何，我都将不离不弃地伴您同行，直至旅途的终点。

进入大陆前，请您收下这份地图。

地 图

第一章 树与二叉树 **1**

1.1 树 1

1.1.1 * 目录树的简单实现 3

1.1.2 树的一些基本概念与性质 5

1.2 二叉树的基本内容介绍 6

1.2.1 二叉树的实现 8

1.2.2 二叉树的遍历 9

1.3 哈夫曼树与哈夫曼编码 23

习题参考答案 **26**

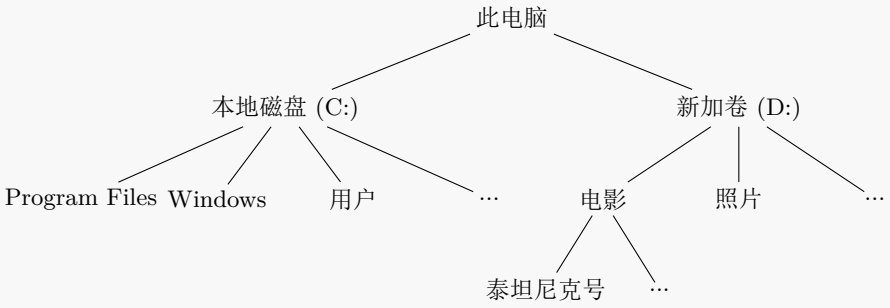
第一章　树与二叉树

1.1　树

在 ?? 节中，我们已经对“什么是树”有了一个感性的认知。与现实中的大树类似，数据结构中的树也是一种由根开始分叉生长的结构。比如游戏中一个队长三个队员的四人小队可以看成一棵树：



你的个人计算机中目录结构的组织是一棵树：



下面给出树的定义。

定义

树是 n（n>=0）个数据元素组成的有限集合，当 n=0 时，称为**空树**。任何一棵非空树都满足以下条件：

1. 有且仅有一个数据元素无前驱，称为**根节点**；
2. 除根节点外，其余每个数据元素都有且仅有一个前驱节点，称为**父节点**；
3. 每个数据元素可以有零个或多个后继节点，称为**子节点**。

由以上定义可知，对于一棵非空树中的任意一个节点，以该节点为根节点，连同该节点的所有后代节点在内所构成的仍是一棵树，称为原树的一棵**子树**。通常认为一棵树本身是自己的子树。很少存在一些场景会关心树中节点的各个子树从左至右是否有次序（即互换后还能不能认为是同一棵树），但通常将节点有次序的树称为**有序树**，节点没有次序的树称为**无序树**。

“树”这个逻辑结构该如何实现？如果采用链表类似的思想，每个节点除保存数据外，再保存所

有指向其孩子节点的指针，像这样：

```
struct TreeNode {
    ElemType data;
    TreeNode* child1;
    TreeNode* child2;
    TreeNode* child3;
    ...
}
```

显然，我们无法预知每个节点有多少孩子节点，因此无法为 TreeNode 结构定义固定数量的孩子节点指针。熟悉 C++ 的同学可能会想到，可以使用一个不定长数组，将节点的所有孩子指针保存到该数组中：

```
struct TreeNode {
    ElemType data;
    std::vector<TreeNode*> childs;  // childs 是一个不定长数组，childs[0], ..., childs[n] 分别表示该节点的
    ↪  各个孩子节点指针
}
```

如图 1.1 所示。这是完全可行，也非常自然的一种树的表示方式。但本书不要求大家熟悉 C++，

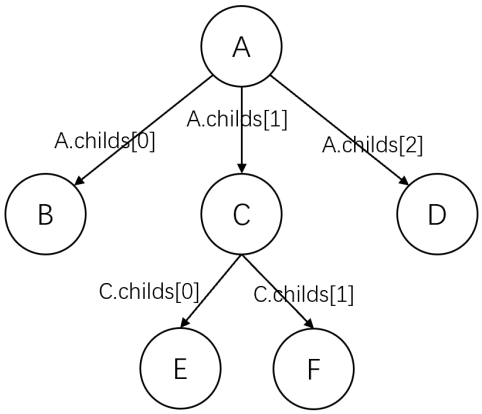


图 1.1 用不定长数组表示树的孩子

因此我们给出树的另一种表示方法，也是更高效的**孩子-兄弟表示法（First Child - Next Sibling Representation）**。该方法存储每个节点的第一个孩子的指针，以及下一个兄弟的指针：

```
struct TreeNode {
    ElemType data;
    TreeNode* first_child;
    TreeNode* next_sibling;
};
```

如图 1.2 所示。

显然该表示法下，只要拿到根节点指针，就能找到树中任何一个节点。因此，可以用根节点指针表示一棵树。如：

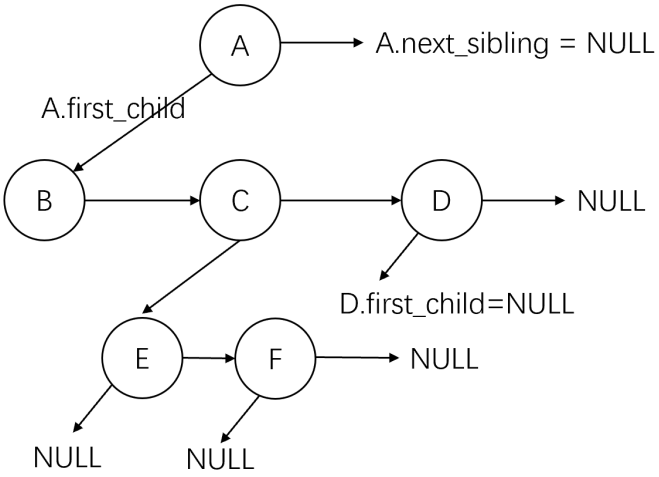


图 1.2 树的孩子-兄弟表示法

```
TreeNode* root;
```

root 这个对象既可以理解为树的根节点的指针，又可以理解为以 root 节点为根的一棵树。

提示

图 1.1 与图 1.2 逻辑上是同一棵树，只不过物理指针的含义不同。

有了以上的内容介绍，我们来做一点有意思的事情（当然你大可以放心地跳过）：用孩子-兄弟表示法尝试实现操作系统的目录树。

1.1.1　* 目录树的简单实现

我们已经知道，操作系统对目录的管理其实就是一棵树。用户随时可能在任何目录新建目录、删除某个目录，我们暂且不考虑文件。并假设目录节点的“数据”只有一个字符串类型的成员，表示目录的名字，如“电影”等。以下代码实现了简单的新建目录、删除目录的操作，具体过程请结合注释。

```
// 在目录 Dad 中，新建一个名为 name 的目录
void DirInsert(TreeNode* Dad, char* name) {
    // 函数实现：创建一个新节点，将其加入 Dad 的孩子链表末尾
    TreeNode* new_node = (TreeNode*)malloc(sizeof(TreeNode));
    strcpy(new_node->name, name);
    new_node->first_child = NULL;
    new_node->next_sibling = NULL;

    // Dad 原本是空目录，则需要更改 first_child 指针。
    if(Dad->first_child == NULL) {
        Dad->first_child = new_node;
        return;
    }

    // 原本不是空目录，修改 next_sibling 指针。
```

```
    TreeNode* last_child = Dad->first_child;
    while(last_child->next_sibling != NULL) {
        last_child = last_child->next_sibling;
    }
    last_child->next_sibling = new_node;
    return;
}

// 销毁一个节点及其所有后代
void DestroyTree(TreeNode* root) {
    if(root == NULL) return;
    // 先递归销毁其子树
    DestroyTree(root->first_child);

    // 再释放根节点
    free(root);
}

// 在目录 Dad 中，删除目录 to_del
void DirDelete(TreeNode* Dad, TreeNode* to_del) {
    // 函数实现：先把 to_del 从父节点的孩子链表中摘掉，再销毁以 to_del 为根的整棵树
    if(Dad == NULL) {
        return;
    }
    // 要删除的就是第一个孩子节点，需要更新 first_child 指针
    if(to_del == Dad->first_child) {
        Dad->first_child = to_del->next_sibling;;
        // 注意这里不能简单地 free(to_del)，因为该目录中可能还有若干目录，目录中可能还有目录，因此要递归销毁。
        DestroyTree(to_del);
        return;
    }

    // 找到要删除的节点的前一个兄弟节点
    TreeNode* pre = Dad->first_child;
    while(pre->next_sibling != to_del) {
        pre = pre->next_sibling;
    }

    // 修改链条，类似单链表的删除操作
    pre->next_sibling = to_del->next_sibling;
    // 注意同样要递归销毁
    DestroyTree(to_del);
    return;
}
```

提示

以上例子也给了我们一个重要提示：树中每个节点的子节点很多时候不能单纯的看成一个节点，而是要看成一棵子树。因此与树相关的操作，很多都会涉及到递归，这一点非常重要。

1.1.2　树的一些基本概念与性质

我们回到逻辑上的数据结构：树。树中有一些大家常用的专业术语与性质，我们在这里列出。

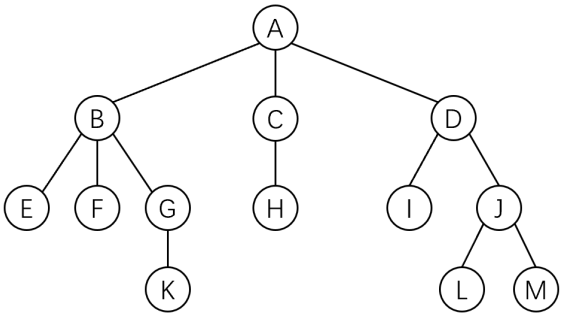


图 1.3

a）树中，一个节点拥有的子树数量称为**节点的度**。一棵树中所有节点的度的最大值称为这棵**树的度**。

b）树中度为 0 的节点称为**叶子节点**，简称**叶节点**。显然叶节点没有孩子节点。不是叶子节点的节点称为**非叶节点**。

图 1.3 中，节点 A、B 的度均为 3，整棵树的度为 3；节点 E、F、K、H、I、L、M 的度为 0，是叶子节点。

c）可以看出树是一种具有层次结构的数据结构。根节点处于最高层，通常定义根为第 1 层，根的孩子为第 2 层。若某节点处在第 i 层，其子树的根节点就处在第 i+1 层。

d）**节点的深度**通常定义为节点的所在层次；而**节点的高度**通常是指从最底层叶子节点（高度为 1）开始逐层累加至该节点时的高度值。所有节点的深度的最大值称为**树的深度**或**树的高度**。即

$$\text{树的高度} = \max\{\text{所有节点的深度值}\} = \max\{\text{所有节点的高度值}\}$$

图 1.3 中，节点 A 的深度为 1，高度为 4；节点 B 的深度为 2，高度为 3；节点 K 的深度为 4，高度为 1。整棵树的深度，或称整棵树的高度，为 4。

提示

1. 有些人喜欢将根节点的高度定义为 0，本书若无特别说明，默认按照根节点高度为 1 来定义。
2. 注意区分节点与树的深度和高度：节点的高度和节点的深度通常不是一回事；而树的高度和树的深度一定是相等的。

e）类似现实中的说法，数据结构中多棵树同样可以构成**森林**。

树的重要性质

我们首先给出一个重要性质：对于任何一棵非空树，其节点数 n 与边数 e 一定有

$$n = e + 1$$

成立。显然，当 $n = 1$ 时，树只有一个根节点，上式显然成立。此后，如图 1.4 所示，从任何一个节点引出一条新的边，都会使得边数 e 和节点数 n 同时加一，因此等式仍然成立。另外，不难发现，每

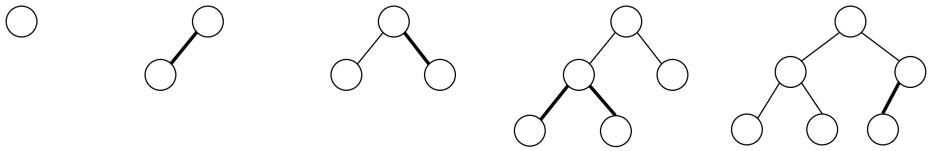


图 1.4

从节点引出一条新的边，会使边数加一的同时，会使该节点的度数加一，所以不难得知，所以对任何一棵非空树，所有节点的度数之和即等于该树的总边数 e。

整理以上两个结论，有：对任何一棵非空树，都有以下结论成立：

树的边数 e = 树中所有节点的度数之和 = n - 1（n 为树的节点数）

由

以上的思维过程也可以反推出一件事：树中度为 n 的节点，会为树“贡献”n 条边。比如：一棵度为 3 的树，若已知度为 3、2、1 的节点分别有 7、5、8 个，则该树的边数一定是

$$7*3+5*2+8*1=39$$

于是可知该树的节点数为 $39+1=40$，于是叶节点的数量就是 $40-7-5-8=20$ 个。

1.2　二叉树的基本内容介绍

简单地说，“最多两个分叉的树”就是二叉树。一棵普通的二叉树本身通常难以在实际问题中应用上，但是它却是后面一些非常有趣的特殊二叉树（如**二叉查找树**、**哈夫曼树**、**堆**等）的理论基础。所以我们有必要首先进行一些理论铺垫。

定义

二叉树要么是一棵空树，要么是一棵：

由根节点以及它的两棵子树构成的树，这两棵子树有左右位置之分，分别称为**左子树**和**右子树**，并且左子树和右子树依然是一棵二叉树。

可以看到，二叉树是**递归定义**的，即用二叉树自身来定义二叉树。如图 1.5 所示的几棵树都是二叉树，根据图中文字介绍，大家也能进一步体会二叉树的“递归定义”。请注意，由图 1.5 我们可以看出，二叉树的递归定义中，递归边界就是二叉树为一棵空树，这一点对我们二叉树中的诸多编程思想尤其重要，大家在后面将会慢慢体会到。显然二叉树的度只可能是 0、1 或 2，二叉树中每一个节点的度也只可能是 0、1 或 2。

提示

“二叉树”与“度为 2 的有序树”有如下区别：

1. 二叉树的度未必是 2，还可能是 0 或 1；
2. 二叉树即使只有一个孩子节点，该节点也要区分左右。而度为 2 的有序树，其“有序”是兄弟之间的位置有序，若某个节点是“独生子女”，则其位置是唯一确定的，不分左右。二叉树

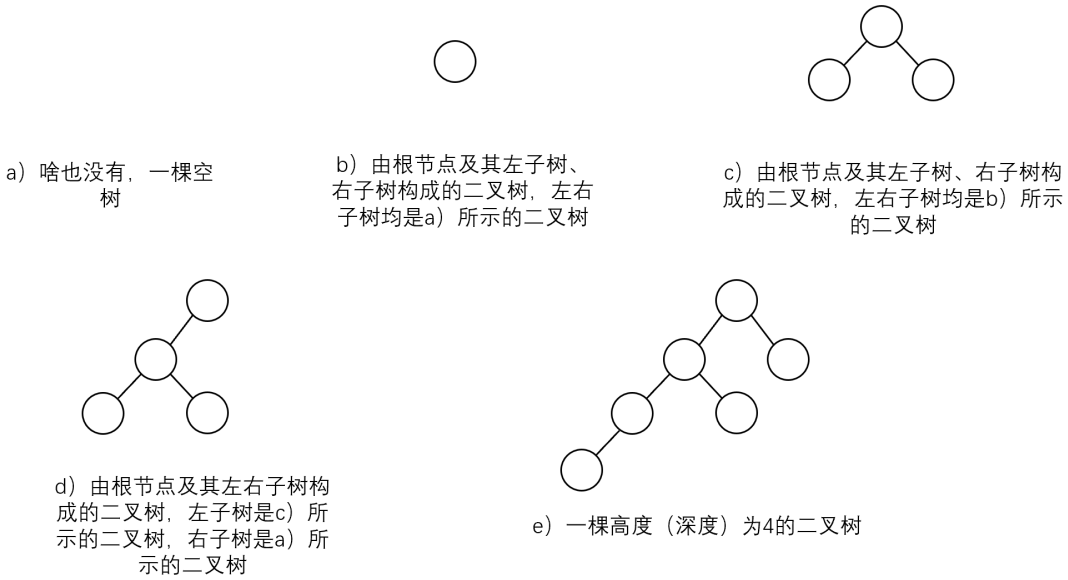


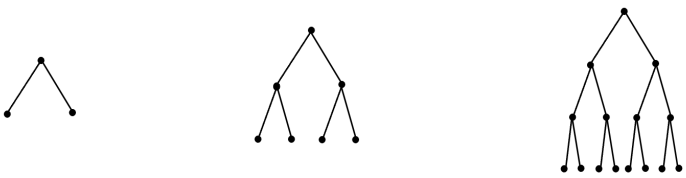
图 1.5 几棵不同的二叉树

> 明确区分孩子的左右位置，这是因为在更多的场景下这是有用的，在后面的内容中我们将有所体会。

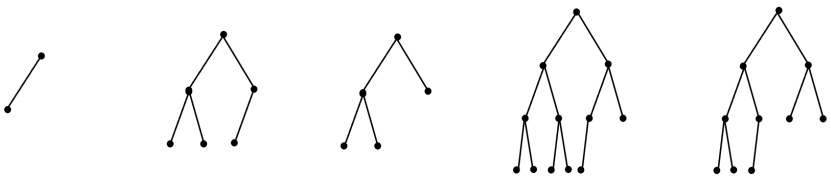
有两种比较丰满的二叉树需要特别关注：满二叉树与完全二叉树。

如果二叉树中的任意一层节点数都达到了当层能达到的最大节点数，这种二叉树称为**满二叉树**；如果除最下面一层外，其余层都达到了当层最大节点数，并且最后一层的节点都一个挨一个地挤在左边，则二叉树称为**完全二叉树**。显然满二叉树一定都是完全二叉树。

图 1.6 展示了一些满二叉树与完全二叉树。



a）一些满二叉树



b）一些完全二叉树

图 1.6 一些满二叉树与完全二叉树

1.2.1　二叉树的实现

将二叉树存储为二叉链表

有了链表的思维，相信大家非常自然地就能想到二叉树的实现方式：树的节点设两个指针分别指向其左孩子和右孩子如下所示：

```
struct BTreeNode {
    ElemType data;
    BTreeNode* lchild;  // 左孩子的指针
    BTreeNode* rchild;  // 右孩子的指针
};
```

这正是二叉树最为经典的实现方式，这种物理结构也称为**二叉链表**，它与普通的单链表相比“多分出了一个叉”。

正如本节之初所指出，二叉树的实际应用在于一些有趣的特殊二叉树。人们对队列、栈等数据结构有“入队、出队、入栈”等常见操作的定义，是因为它们切实有用，有重要的实际意义，而普通二叉树几乎不存在实际应用，因此通常不会为它定义一些常见操作，以上介绍的理论，是为了后面的特殊二叉树服务。此处如果你感兴趣，可以尝试完成下面的“思考”。

思考

建立一棵你中意的二叉树，并遍历这棵树，打印所有节点的数据内容。

二叉树的顺序存储

基于二叉树有上下的层次关系以及左右的次序关系，如果我们从上到之下、从左到右地为二叉树的每一个节点依次编号，就可以将这些节点依次存储在数组中的对应下标之处，如图 1.7 所示，我们这里遵循多数人的习惯，根节点从 1 开始编号，即从数组下标 1 处开始存储，而非从 0。根据规律不难得出，**在这种存储方式下，任一节点 i，其父节点的编号一定是 $\lfloor \frac{i}{2} \rfloor$（前提是 i>1，即该节点有父节点）；并且其左孩子如果存在的话，编号一定是 2i，右孩子如果存在的话，编号一定是 2i+1。**

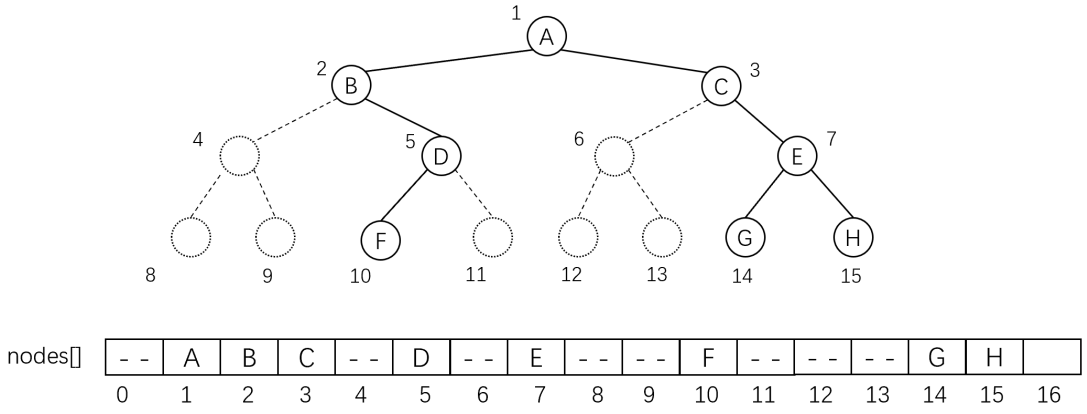


图 1.7 二叉树的顺序存储

这种表示法可以由以下代码实现：

```
struct BTreeNode{
    ElemType data;
    int valid; // 该节点是否存在
};
BTreeNode nodes[MaxSize];  // 该数组中保存的就是二叉树的所有节点
```

提示

一般情况下每个节点还需要一个字段 valid 标识是否有效。但如果你存放的数据满足某种特点，比如一定都是正整数，那么你也可以不要 valid 字段，而是通过将数据的值置为，如-1，来表示该位置处节点不存在。

二叉树的这种存储方式的问题在于，即使某个节点不存在，也要浪费一个编号位，最坏情况下，当二叉树形状趋于深度为 h 的一条直线时，我们用约 2^h 个空间仅存储了约 h 个数据。

经过观察不难发现，**该表示法对完全二叉树（以及满二叉树）非常友好**。因为完全二叉树的节点从上到下、从左到右是连续的，不存在空缺的编号。在介绍后面的内容时，我们也能看到该表示法在完全二叉树中的妙用。

在进入本章真正有趣的部分之前，我们还要进行一场最后的理论准备：二叉树的遍历。

1.2.2　二叉树的遍历

遍历一个顺序表有多种方式，可以从前向后，也可以从后向前等。类似地，遍历二叉树也有多种方式。常见的、有意义的遍历方式通常有如下 4 种：**层序遍历**、**先序遍历**、**中序遍历**、**后序遍历**。相信大家最容易想到的遍历方式就是从上到下、从左到右地一个一个节点地访问，这就是**层序遍历**。对于图 1.8 中所示的二叉树，层序遍历得到的访问序列是 A,B,C,D,E,F,G,H,I,J。

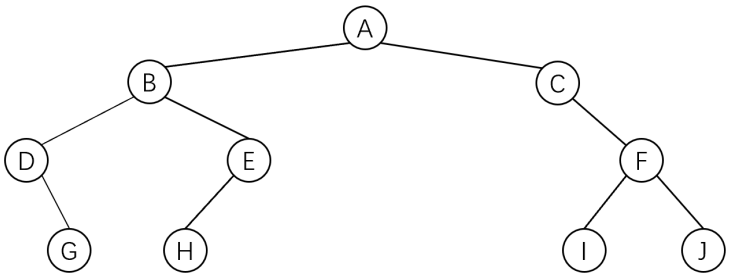


图 1.8

提示

我们与多数教材保持一致，以二叉链表存储的二叉树为例，给出二叉树的遍历方式，这也是应用得更多的场景。

二叉树的层序遍历

对于链式存储的二叉树：

```
struct BTreeNode {
    ElemType data;
    BTreeNode* lchild;  // 左孩子的指针
    BTreeNode* rchild;  // 右孩子的指针
};
```

如何从上到下，从左到右地依次访问每个节点？显然我们仅通过二叉树已有的物理结构是不行的，以图 1.8 为例，当我们从 A 走到 B 时，我们知道下一步要访问 C，但却没有一种有效的手段优雅地实现接下来的操作。所以我们需要一种手段，每走到一个节点之时，先将其两个孩子按正确的顺序保存起来，它们就是接下来即将被访问的节点。

承先贤之智，将下法传与思渡者：可使用一个队列，将更上层的节点优先入队，并且确保同层级的节点中，更左边的优先入队。这样，队中节点顺序即是层序遍历的顺序，依次出队访问即可。具体来说，实现二叉树层序遍历的过程如下：

1） 将根节点 root 加入队列 q；

2） 出队访问队首节点 front_node；

3） 如果 front_node 有左孩子，先将左孩子入队；

4） 如果 front_node 有右孩子，再将右孩子入队；

5） 若队列 q 不空，回到第 2）步；否则说明遍历完成，算法结束。

显然第一步将根节点加入队列是毋庸置疑的。而第 3）、4）两步，先加入左孩子再加入右孩子，则保证了从左到右的访问顺序。

*** 思维拓展**

如果你希望从上到下、从右到左地遍历二叉树，将上面的 3）、4）两步交换位置即可。

层序遍历的伪代码如下所示。

```
// 对根为 root 的二叉树进行层序遍历。对每个节点执行 visit() 函数。
    void LayerOrderTravel(BTreeNode* root) {
        if(root == NULL) return ;
        Queue q;
        InitQueue(q);
        EnQueue(q, root);  // 根节点入队
        while(!Empty(q)) {
            BTreeNode* front_node = NULL;
            DeQueue(q, front_node);    // 队首节点出队，赋值给变量 front_node
            visit(front_node);         // 访问 front_node
            if(front_node->lchild != NULL) {
                EnQueue(q, front_node->lchild);    // 入队左孩子
            }
            if(front_node->rchild != NULL) {
                EnQueue(q, front_node->rchild);    // 入队右孩子
            }
        }
    }
```

表 1.1: 二叉树层序遍历的执行过程

步骤	出队节点	访问节点	新入队节点	操作完成后的队列（队首 → 队尾）
初始	—	—	A	A
1	A	A	B，C	B，C
2	B	B	D，E	C，D，E
3	C	C	F	D，E，F
4	D	D	G	E，F，G
5	E	E	H	F，G，H
6	F	F	I，J	G，H，I，J
7	G	G	—	H，I，J
8	H	H	—	I，J
9	I	I	—	J
10	J	J	—	空

对于图 1.8 中所示的二叉树，其详细的层序遍历过程如下表 1.1 所示。可以看出，正是队列的先进先出特性，保证了节点按照从上到下、同层从左到右的顺序被访问。

除层序遍历外，对一棵二叉树的根节点、左子树和右子树也可以有不同的访问顺序。访问一棵二叉树时：

1. 若先访问根节点，再访问左子树，最后访问右子树，这种“根-左-右”的遍历方式称为**先序遍历**，也称**先根遍历**。
2. 若先访问左子树，再访问根节点，最后访问右子树，这种“左-根-右”的遍历方式称为**中序遍历**，也称**中根遍历**。
3. 若先访问左子树，再访问右子树，最后访问根节点，这种“左-右-根”的遍历方式称为**后序遍历**，也称**后根遍历**。

提示

请注意，无论何种访问方式，左子树一定是先于右子树访问。所谓的“先、中、后”指的是“根节点”被“先”访问，还是“中”或“后”访问。

*** 思维拓展**

注意：本内容可能带来一些认知和思维上的启发，但对思渡者来说，可能带来知识记忆上的混乱。

我们为什么不将“根右左”、“右根左”、“右左根”分别定义为先根、中根、后根遍历？这是因为，二叉树的左右子树具有天然的对称性。将左与右交换，树的结构和**本质**并没有发生变化，它描述的还依然是那个“东西”，只是我们人类需要换一个方向去观察和描述它。

比如你将左边视为头时，[3,5] 的**本质**可以理解为“3 在前，5 在后的两个数”；而事实上 [5,3] 同样可以描述“3 在前，5 在后的两个数”，只需要你把右边视为头，或者把“前”定义为后。二者本质相同，只是我们人类观察和理解它的视角不同。

更进一步地讲，“前”与“后”、“左”与“右”的方向本身就是人为定义的，左子树之所以叫左子树，只是因为我们把“左”定义为了左而已。“左”与“右”本质上并无区别。

我们约定先左，只是因为人类的习惯就是左在前，右在后，这更符合绝大多数人的思维方式。

如果你没有阅读上述“思维拓展”，请直接跳过。如果你阅读了，请放声朗读以下内容：

先序遍历根左右 根左右 根左右； 先序所以根在先；

中序遍历左根右 左根右 左根右； 中序所以根在中；

后序遍历左右根 左右根 左右根； 后序所以根在后。

左一定要先于右； 左一定要先于右； 左一定要先于右；

先序遍历根左右 根左右 根左右； 先序所以根在先；

中序遍历左根右 左根右 左根右； 中序所以根在中；

后序遍历左右根 左右根 左右根； 后序所以根在后。

左一定要先于右； 左一定要先于右； 左一定要先于右；

二叉树的先序遍历

参照图 1.9，先序遍历下，对任何一棵二叉树 BT，总是先访问其根节点，再访问其左子树 LBT 和右子树 RBT。递归地，对于 LBT 和 RBT 又有同样的规则。因此递归遍历便是很自然的想法，而递归边界就是某棵子树为空树。先序遍历的递归代码如代码 1.1 所示：

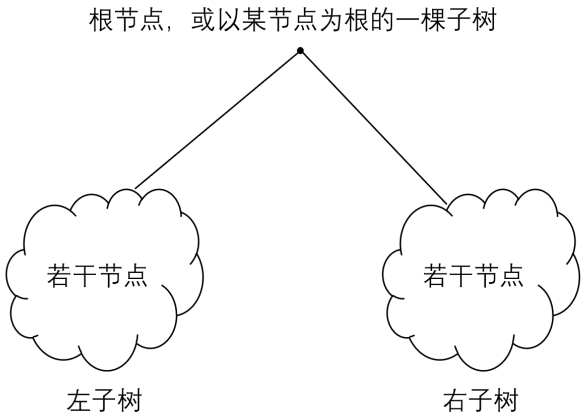


图 1.9

代码 1.1

```
1 void PreOrderTravel(BTreeNode* root) {
2     if(root == NULL) return ;  // 递归边界
3     visit(root);   /* 先序遍历，先访问根节点。请注意：这里 root 应理解为递归过程中访问的每棵子树的根
      ↪  节点，不能仅简单地理解为整棵树的根节点。*/
4     PreOrderTravel(root->lchild);  /* 递归遍历左子树。请注意：一定不能写成 visit(root->lchild)。此处场
      ↪  景要把 root->lchild 看成一棵左子树，而不能仅仅看成一个节点。*/
```

```
5     PreOrderTravel(root->rchild);   // 递归遍历右子树
6 }
```

维火女

亲爱的余灰大人，如果您思渡至此，已经通晓了先序、中序及后序的遍历过程，请握紧我的手，我将带您传送至—waypoint-4—（第 13 页）处。

我们以图 1.8 模拟一遍整个过程：

暂且拟用 Tree(T) 表示以 T 为根节点的子树。则先序遍历过程如下：

走到 Tree(A) -> 先根遍历，因此直接访问 A；然后访问 A 的左子树，所以

走到 Tree(B) -> 同样地，直接访问 B；然后访问 B 的左子树，

走到 Tree(D) -> 访问 D；然后先访问 D 的左子树，再访问 D 的右子树。先

走到 Tree(NULL，即 D 的左孩子)，为空树，递归边界，返回到 D；该访问 D 的右子树了：

走到 Tree(G，即 D 的右孩子)，访问 G；

走到 Tree(NULL, G 的左孩子)；返回到 G；

走到 Tree(NULL, G 的右孩子)；返回到 G；

返回到 D；

返回到 B；B 的左子树由上面的过程已经访问完成，该访问 B 的右子树了。

走到 Tree(E)；访问 E；

走到 Tree(H)；访问 H；我们不再赘述空子树的访问过程。

返回到 E-> B -> A；此时 A 的左子树已经全部访问完成，根据先序规定的访问顺序：①A ②A 的左子树 ③A 的右子树，A 及 A 的左子树已访问完成，接下来访问 A 的右子树；

走到 Tree(C)；访问 C；

走到 Tree(F)；访问 F；

走到 Tree(I)；访问 I；

走到 Tree(J)；访问 J；

一路回退到 A；代码结束。先序遍历得到的序列为 A,B,D,G,E,H,C,F,I,J。

waypoint-4

可以看出，先序遍历的思想，简单说就是：

1. 访问根节点（先根）；
2. 递归地遍历左子树；
3. 递归地遍历右子树。

后面的中序、后序遍历，实际上也是这三步，只不过访问根节点的顺序有所不同。

请注意，根据先序遍历一定最先访问根节点的特性，对一棵二叉树的先序遍历序列，序列的第一个节点一定是根节点。如上例中，整棵树的遍历序列是

$$A, B, D, G, E, H, C, F, I, J$$

因此 A 是整棵树的根节点。而对于 A 的左子树中的五个节点，遍历序列是

$$B, D, G, E, H$$

因此 B 是该子树的根节点。再如以 F 为根节点的子树，遍历序列是

$$F, I, J$$

因此 F 是根节点。

二叉树的中序遍历

参照图 1.9，中序遍历总是先访问左子树，再访问根节点，最后访问右子树。中序遍历的思想与先序遍历类似，只不过在访问根节点之前，要先递归遍历左子树：

1. 递归遍历左子树；
2. 访问根节点（中根）；
3. 递归遍历右子树。

可以看到，先序、中序、后序在代码上的区别其实就是把“访问根节点”这一操作放在哪一步的区别。中序遍历的代码如下所示。

```
void InOrderTravel(BTreeNode* root) {
    if(root == NULL) return ;   // 递归边界
    InOrderTravel(root->lchild);
    visit(root);        // 中根
    InOrderTravel(root->rchild);
}
```

我们以图 1.8 模拟一遍整个过程。拟用 Tree(T) 表示以 T 为根节点的子树。则先序遍历过程如下：

走到 Tree(A) -> 中根遍历，因此先访问 A 的左子树，再访问 A，最后访问 A 的右子树。所以暂且不访问 A，而是先

走到 Tree(B) -> 同样地，先访问 B 的左子树：

走到 Tree(D) -> 同样地，先访问 D 的左子树，因此

走到 Tree(NULL，即 D 的左孩子) -> 为空树，递归边界，返回到 D；该访问 D 这个节点了：

访问 D，此时 D 及 D 的左子树均访问完成，该访问 D 的右子树了。所以

走到 Tree(G) -> 依旧先左；走到 Tree(NULL, G 的左孩子)；返回到 G；

访问 G；此时 G 及 G 的左子树均访问完成，该访问 G 的右子树了。

走到 Tree(NULL, G 的右孩子)；返回到 G；

返回到 D；此时 D 及 D 的左右子树均已经访问完成。

返回到 B；此时 B 的左子树已访问完成，根据中序遍历规则，该访问根节点了，于是

访问 B；此时 B 的左子树及 B 都访问完成，该访问 B 的右子树了。

走到 Tree(E)；走到 Tree(H) -> 访问 H；我们不再赘述空子树的访问过程。

回到 E；访问 E；

回到 B -> A；此时 A 的左子树已经全部访问完成，根据中序规定的访问顺序：①A 的左子树 ②A ③A 的右子树，现在 A 的左子树已经访问完成，所以接下来访问 A，然后访问 A 的右子树。

走到 Tree(C) -> 左子树为空；访问 C；

走到 Tree(F) ->

走到 Tree(I) -> 访问 I；

回到 F；访问 F；

走到 J；

访问 J；

一路回退到 A；代码结束。中序遍历得到的序列为 D, G, B, H, E, A, C, I, F, J

请注意：对于二叉树的中序遍历，根节点的访问总是在左子树和右子树的中间。因此，若已知某棵树的根节点是哪一个，就可以根据根节点在中序序列中的位置，将该中序序列从根节点处砍成两半，得到左子树和右子树的中序遍历序列。如上面的例子，若已知根节点为 A，则立即可以得到：
① 根节点左子树的中序遍历序列是

$$D, G, B, H, E$$

② 根节点右子树的中序遍历序列是

$$C, I, F, J$$

二叉树的后序遍历

有了前面的内容，后序遍历就变得很好理解。后序遍历是“左-右-根”，即根节点的访问在最后：

1. 递归遍历左子树；
2. 递归遍历右子树；
3. 访问根节点（后根）。

代码如下。

```
void PostOrderTravel(BTreeNode* root) {
    if(root == NULL) return ;   // 递归边界
    PostOrderTravel(root->lchild);   // 先递归遍历左子树
    PostOrderTravel(root->rchild);   // 再递归遍历右子树
    visit(root);        // 左子树、右子树均访问完成后，最后访问根节点。
}
```

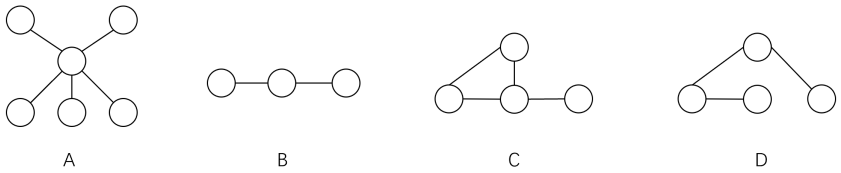
先序遍历是最先访问根节点，而后序遍历是最后访问根节点。因此对后序遍历序列来说，序列中的最后一个节点一定是根节点。对于图 1.8 所示的二叉树，其后序遍历得到的序列为 G,D,H,E,B,I,J,F,C,A。

—— BONFIRE HIT ——

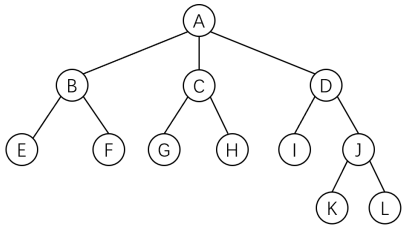
此处提示大家，二叉树这里的题目复杂多变，思维难度较大。

习题 1　（参考答案见习题 1 参考答案，第 27 页）

1. 下图中，不属于**树**的结构是 _____。



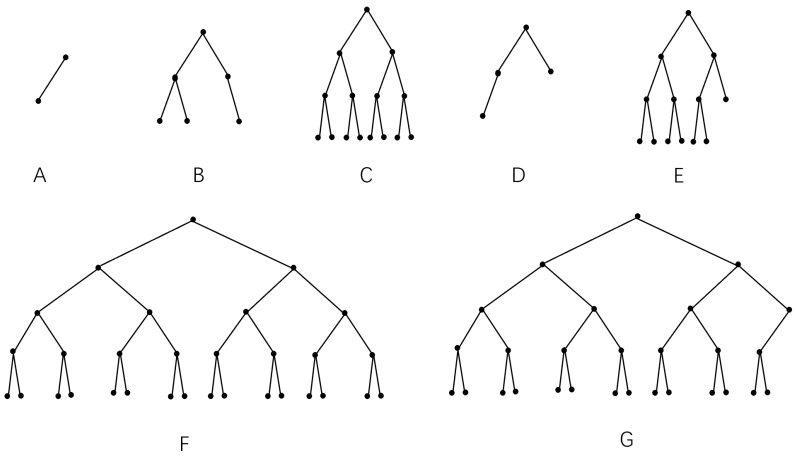
2. 对于下图所示的树，下列说法错误的是 _____。



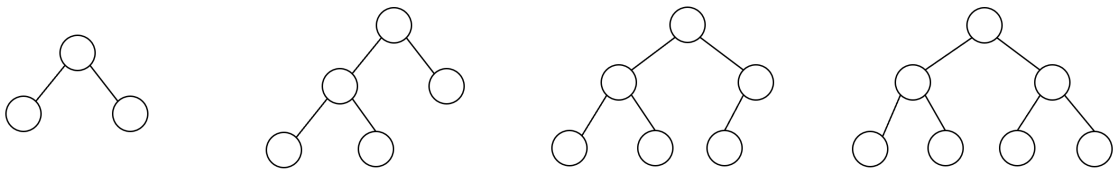
A. 节点 B 的度为 2　　B. 节点 A 的度为 3
C. 这是一棵二叉树　　D. 这棵树的度为 3

3. 关于上题图所示的树，下列说法错误的是
A. 若根节点高度定义为 1，则该树的高度为 4
B. 若根节点高度定义为 1，则该树的深度为 3
C. 节点 F 的度为 0，是叶子节点
D. 以 B、C、D 为根节点的子树的集合构成了一个森林

4. 下图中，是满二叉树的有 _____；不是满二叉树，但是是完全二叉树的有_____。



5. 请观察下列四棵二叉树，分别统计每棵树中度为 0 的节点（叶节点）和度为 2 的节点的数量，并将结果填入表格。



图一　图二　图三　图四

	度为 0 的节点数 (记为 n_0)	度为 2 的节点数 (记为 n_2)
图一		
图二		
图三		
图四		

请你思考：

1）对于这四棵树，度为 0 的节点数与度为 2 的节点数之间有什么关系？

2）这种关系是否可能对任何非空二叉树都成立？请尝试画出其他形状的二叉树，检验你的猜想。

3）不必从数学角度严谨证明，你能否尝试解释，为什么这两个数量之间会存在这样的关系？

6. 一棵非空二叉树中，有 8 个度为 2 的节点，3 个度为 1 的节点。请问：

1）能否确定这棵二叉树有多少个叶节点？

2）能否确定这棵二叉树一共有多少个节点？

7. 一棵非空二叉树共有 20 个节点，其中有 7 个度为 1 的节点。请问这棵二叉树的叶节点数可能是多少？

8. [多选] 若完全二叉树有 n（n > 0）个节点，从上到下，从左到右分别编号 1 至 n，则下列说法中正确的有_____

A. 若某节点不是根节点，其编号为 i，则其父节点的编号为 $\lfloor \frac{i}{2} \rfloor$

B. 若某节点编号为 i，且 2 * i > n，则节点 i 一定是叶子节点

C. 树中度为 1 的节点数可能大于 1

D. $\lfloor \frac{n}{2} \rfloor$ 一定是所有非叶子节点中，编号最大的那个

9. 思考：二叉树的第 i 层上（设根为第 1 层）至多有多少个节点？

10. 思考：二叉树的叶子节点可能出现在哪些层？完全二叉树呢？满二叉树呢？

11. （某年 408 真题）已知一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶节点，则该完全二叉树的节点个数最多是_____

A. 39　B. 52　C. 111　D. 119

12. （某年 408 真题）若一棵完全二叉树有 768 个节点，则该二叉树中叶节点的个数是_____

A. 257　B. 258　C. 384　D. 385

13. （某年 408 真题）对于任意一棵高度为 5 且有 10 个节点的二叉树，若采用顺序存储结构保存，每个节点占 1 个存储单元（仅存放节点的数据信息），则存放该二叉树需要的存储单元数量至少是_____

A. 31　B. 16　C. 15　D. 10

14. （某年 408 真题）在一棵度为 4 的树 T 中，若有 20 个度为 4 的节点，10 个度为 3 的节点，1 个度为 2 的节点，10 个度为 1 的节点，则树 T 的叶节点个数是_____

A. 41　B. 82　C. 113　D. 122

15. 由于二叉树的链式存储本质上是二叉链表，所以某同学想到了用静态链表存储二叉树。该同学设计如下：

代码 1.2

```
struct node {
    ElemType data;
    int lchild;
    int rchild;
};
struct node nodes[MaxSize];
```

其中，nodes 是一个用于存储二叉树节点的数组，lchild 和 rchild 分别用于存储左、右孩子节点在数组中的下标；若相应的孩子节点不存在，则使用 -1 表示。根节点存放于数组下标 1 处。下列说法中正确的是_____

A. 该存储方式下，若某节点存放在下标 i 处，则其左孩子（若存在）一定存储在下标 2i 处。
B. 该存储方式下，同一个节点的所有孩子节点在物理存储上一定是紧挨着的
C. 该存储方式通常不适用于保存最大节点数量不确定的二叉树
D. 假设节点数为 n，则所有节点从上到下、从左到右地依次存放于数组下标 1-n 处。

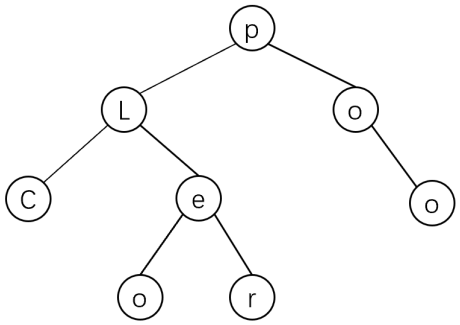
16. 设某二叉树 T 保存 int 类型的数据，采用二叉链表存储：

```
struct node {
    int data;
    node* lchild;
    node* rchild;
};
```

1）请你编写一个算法，将 T 中所有值为 5 的元素改为 10，算法返回 void 即可。

2）若算法要求：当修改了至少一个元素时返回 1，否则返回 0，请你思考该如何实现，有条件的话可以写出代码。

17. 某二叉树 T 保存 char 类型的数据，如下图所示。

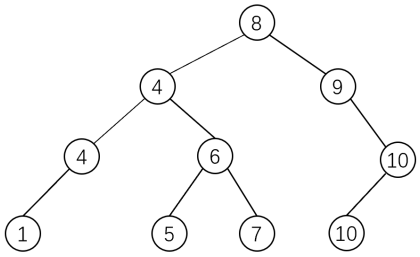


该树采用二叉链表存储：

```
struct node {
    char data;
    node* lchild;
    node* rchild;
};
```

1）请你写出后序遍历该树 T 的代码，打印每个节点的值。
2）打印出的字符序列是什么？

18. 设某二叉树 T 保存 int 类型的数据，如下图所示。



该树采用二叉链表存储：

```
struct node {
    int data;
    node* lchild;
    node* rchild;
};
```

1）观察这棵树，该二叉树中，任意一个节点的左子树与该节点存在什么关系？右子树与该节点存在什么关系？
2）如果对这棵树进行中序遍历，得到的序列是什么？该序列有何特点？
3）你能否写出一个函数，功能是在这棵树中插入一个新的值为 e 的数据，并且加入新节点后，这棵树依然具有第 1）问及第 2）问中的特性？

```
void InsertTree(node* &root, int e) {

}
```

有了前面的铺垫，相信你对树与二叉树的基本理论内容已经有了比较深刻的认识，并且对树中的递归代码建立起了一定的感觉。接下来就让我们进入本章中真正有趣（但也有一定难度）的部分，来看一些有实际应用价值的特殊的树。

1.3　哈夫曼树与哈夫曼编码

请思渡者考虑以下最为经典的“合并柴薪”问题：

在初始火焰的熔炉周围，散落着 n 堆柴薪，重量分别为 $w_1, w_2, ..., w_n$，你每次可以搬运任意两堆柴薪 i 和 j，将它们合成一堆重量为 $w_i + w_j$ 的新柴薪，这个过程需要消耗的体力为 $w_i + w_j$。显然，$n - 1$ 次搬运之后，就只剩下一堆柴薪了。请问你能否设计最优的合并次序方案，使得耗费的体力最少？并给出消耗的体力值。

例如有 3 堆柴薪，重量分别为：

$$[3, 6, 11]$$

第一次，你将 6 和 11 合并到一起，消耗体力 17，还剩两堆柴薪：

$$[3, 17]$$

第二次，你将 3 和 17 合并到一起，消耗体力 20，只剩一堆柴薪：

$$[20]$$

合并过程一共消耗了 17 + 20 = 37 点体力。显然，若你先合并 3 和 6，则可以达成更少的体力消耗：(3+6)+(9+11)=29 点体力。

我们看一个稍微复杂一点的例子，五堆柴薪重量分别是：3, 6, 2, 3, 4。我们先随意将它们合并，看看过程中会发生什么。合并过程可以如图 1.10 所示：

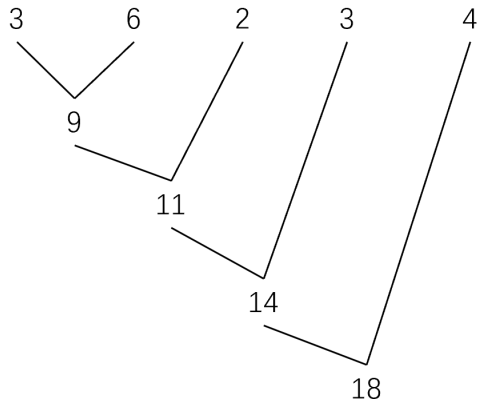


图 1.10 柴薪的合并过程

可以看出，一堆柴薪参与合并后，并不会从后续的合并过程中消失，而是会作为新柴薪的一部分，继续参与后续合并。例如，初始重量为 3 的柴薪与重量为 6 的柴薪合并后，形成一堆重量为 $9 = 3 + 6$ 的柴薪。虽然新柴薪的重量变成了 9，但其中仍然包含原来重量为3 的柴薪（9=3+6 嘛）。因此，当这堆重量为 9 的柴薪再次参与合并时，产生的消耗中仍然包含原来那 3 的重量。也就是说：**每堆原始柴**

薪在每次参与合并时，都会为总消耗贡献一次自己的初始重量。于是，若初始重量为 w_i 的柴薪参与了 k_i 次合并，那么它对总消耗的贡献就是 w_ik_i。因此，整个合并过程的总消耗可以表示为

$$W = \sum_{i=1}^{n} w_i k_i$$

另一方面，由图 1.10 可以看出，原始柴薪越靠后参与合并，其合并次数就越小，如最后的 4 只参与了 1 次合并，而第一个 3 参与了 4 次合并。因此，自然要将原始重量更大的柴薪放在更后面合并，这样其参与的合并次数就小，因而总体造成的体力消耗就更小。

基于以上讨论，很容易设计出最优的合并方案：每次从当前所有柴薪中，选出两堆重量最小的进行合并，直至最后合并完成。对于 3, 6, 2, 3, 4 这五堆柴薪，一种最优的合并方式如图 1.11 所示。

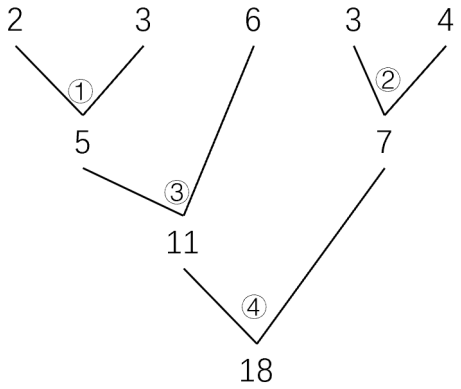


图 1.11 最优合并过程

第①步，将最小的 2 和 3 合并，消耗体力 5，合并后新的柴薪：

$$[5, 6, 3, 4]$$

第②步，将这四堆中最小的 3 和 4 合并，消耗体力 7，合并后新的柴薪：

$$[5, 6, 7]$$

第③步，将最小的 5 和 6 合并，消耗体力 11，合并后新的柴薪：

$$[11, 7]$$

第④步，将最小的 (也是仅剩的)11 和 7 合并，消耗体力 18，合并成最终的一堆柴薪：18。可以看出，消耗的总体力 $= 5 + 7 + 11 + 18 = 41$。换个角度，2、3 贡献了 3 次自己的初始重量、6 贡献了 2 次，3 和 4 贡献了 2 次，因此 $W = \sum_{i=1}^{n} w_i k_i = (2 + 3) * 3 + 6 * 2 + (3 + 4) * 2 = 41$。

相信你早已看出来了，合并的过程其实就构成了一棵二叉树，原始的五堆柴薪就是五个叶子节点，而每堆初始柴薪参与的合并次数 k_i 就是该叶子节点到根节点的**路径长度**。在以上二叉树中，每个节点的数值代表了要搬动这堆柴薪需要消耗的体力。事实上在许多应用中都要类似现象，我们会将树中的节点赋予一个表示某种意义的数值，称为节点的**权**。从树根到一个节点的路径长度与该节点上权值的乘积，就叫做该节点的**带权路径长度 (WPL)**（可以理解为这堆柴薪在后续合并过程中还会贡献的总体力消耗），而树中所有叶子节点的带权路径长度之和就称为这棵树的**带权路径长度 (WPL)**，记为。

$$\mathrm{WPL} = \sum_{i=1}^{n} w_i k_i$$

其中 w_i 是第 i 个叶节点的权值，k_i 是该叶节点到根节点的路径长度。在含有 n 个带权叶节点的二叉树中，带权路径长度最小的二叉树就称为**哈夫曼树**。可以看到在该定义下，“合并柴薪”的问题其实就是求以这 n 堆柴薪为叶节点的哈夫曼树，其带权路径长度是多少。

通过这个例子，我们也理解了“带权路径长度”以及哈夫曼树的意义。图 1.11 所示的过程其实也就是由给定的若干个节点，构造一棵哈夫曼树的过程。这里将该过程完整地叙述如下：

1. 初始化。根据给定的 n 个权值，构造 n 棵仅包含一个节点的二叉树，每个节点的权值分别为 $w_1, w_2, \ldots, w_n$。此时，每棵树都只有一个根节点，且该节点同时也是叶节点。
2. 选择与合并。从当前所有二叉树中，选择根节点权值最小的两棵二叉树，将它们合并为一棵新的二叉树。具体而言，以一个新节点作为根节点，将选出的两棵二叉树分别作为新根节点的左、右子树，并将新根节点的权值设置为这两棵树的根节点权值之和。
3. 更新集合。将刚刚合并得到的新二叉树加入当前的二叉树集合，同时从集合中移除原先被合并的两棵二叉树。这样，集合中的二叉树数量便减少一棵。
4. 重复执行步骤 2 和步骤 3，直到当前集合中只剩下一棵二叉树。
5. 得到哈夫曼树。最终剩下的二叉树就是所构造的哈夫曼树。其所有叶节点的带权路径长度之和最小。

需要注意的是，选择权值最小的两棵树进行合并时，若存在权值相同的节点，可以任选符合条件的两棵树。不同的选择可能构造出形态不同的哈夫曼树，但所得的最小带权路径长度相同。

习题参考答案

习题 1 参考答案

1. 选 C。一个最快的方法就是判断节点是否存在环。根据树的定义，若存在环则节点之间的前驱后继关系一定不符合树的定义。笔者不建议读者在此处逻辑上浪费过多时间。

2. 选 C。根据度的定义知道 A、B、D 正确。二叉树中任何一个节点至多有两棵子树，C 错误。

3. 选 B。树的高度和树的深度一定相等，并且等于树中层次的数量。该图中的树有四层，因此树的高度与深度都是 4，A 正确；D 显然正确。

4. 是满二叉树的有C、F；不是满二叉树，但是是完全二叉树的有A、D、E。完全二叉树只能是在最底层、从右开始有连续的残缺。F 项倒数第二层存在残缺；B 项最底层的残缺并非从右开始并且连续，因此 F、B 都不是完全二叉树。

5. 如下表。

	度为 0 的节点数	度为 2 的节点数
图一	2	1
图二	3	2
图三	3	2
图四	4	3

事实上，**记二叉树中度为 2 的节点的数量为 n_2，度为 0 的节点（即叶节点）数为 n_0，则 $n_0 = n_2+1$ 对任何非空二叉树均成立**。对于数学上严谨的证明，感兴趣的同学可以参考相关资料，我们这里只给出直观上的理解。

相信你也已经看出来了，最初对于只有一个节点的二叉树，$n_0 = n_2+1$ 显然成立。此后，在二叉树中增加节点，只可能从度为 0 的节点“引出边”，或者从度为 1 的节点“引出边”。

1）从一个度为 0 的节点“引出一条边”，不会增加 n_2 的数量，并且 n_0 会被吃掉一个，也会新增一个，相当于没变。所以该操作不影响 n_0 和 n_2 的数量关系；

2）从一个度为 1 的节点“引出一条边”，则会将 n_0 和 n_2 的数量都增加一个。该操作同样不影响 n_0 和 n_2 的数量关系；

所以，只要二叉树中至少有一个节点，此后无论加入多少个节点，n_0 和 n_2 的关系都不会变化。$n_0 = n_2 + 1$ **对任何非空二叉树都成立。事实上这是二叉树中一个非常重要的性质**，很多教材喜欢将此直接作为“二叉树的性质”给出，但笔者认为，探索、发现规律的过程可以锻炼能力、培养创造力和解决问题的能力。并且这对知识的记忆会是一个很强的冲击。如果直接记住公式，非常容易忘，并且很可能出现“n_0 是啥意思来着？是谁等于谁加一来着?”这样的现象。而有了这个过程，当你忘记的时候，拿出笔画几个圈，立即能回忆起来。

6. 由上一题的结论可知，非空二叉树中度为 2 的节点 $n_2 = 8$，则度为 0 的节点数一定是 $8 + 1 = 9$，即叶节点一定是 9 个。题目又说了度为 1 的节点有 3 个，所以二叉树中节点总数为 $n = n_0 + n_1 + n_2 = 9 + 3 + 8 = 20$。

7. 由已知信息可知，有 $n_0 + n_2 = 13$ 成立。又根据前面的结论有 $n_0 = n_2 + 1$ 成立，解二元一次方程，得 $n_0 = 7$。即叶节点有 7 个。

8. 选 A、B、D。对于此类问题，动手画一画图就非常容易得到答案。事实上本题中的几个选项同样可以作为二叉树的性质来记忆：

若完全二叉树有 n（n > 0）个节点，从上到下，从左到右分别编号 1 至 n，则：

a）若某节点编号为 i，且 2 * i > n，则节点 i 一定是叶子节点；

b）对完全二叉树来说，树中度为 1 的节点数要么是 0，要么是 1，绝不可能大于 1。

c）$\lfloor \frac{n}{2} \rfloor$ 一定是所有非叶子节点中，编号最大的那个

此处笔者提醒大家对性质 c）稍作留意。该选项是后面我们介绍堆时会用到的一个完全二叉树的性质。也可以利用该性质快速解决一些问题。

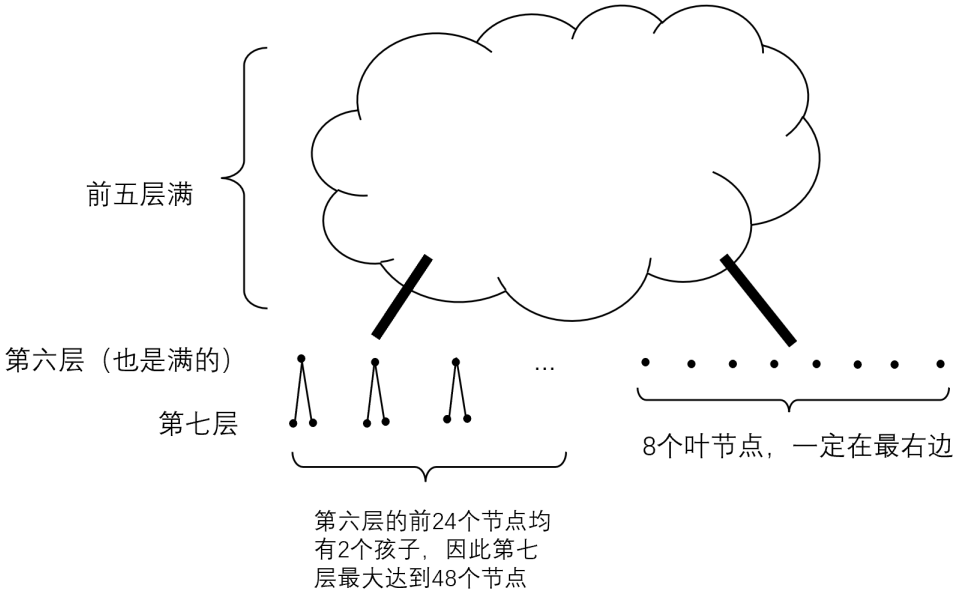
9. 各层及其最大节点数有如下规律：

层数	最大节点数
1	1
2	2
3	4
4	8
5	16
⋮	⋮

由数学归纳法可以证明，第 i 层的最大节点数为 2^{i-1}。该结论可以作为二叉树的性质来记忆：**二叉树中第 i 层的最大节点数为 2^{i-1}**。

10. 随手画画图，就很容易发现结论：对于二叉树，其叶子节点可能出现在任何一层；**对于完全二叉树，叶子节点仅可能出现在最底层或者自下而上的第二层**；对于满二叉树，叶子节点仅可能出现在最底层。完全二叉树叶子节点仅可能出现在最下两层这一性质会带来一些有趣的问题，我们已在后面的题目中给出。

11. 选 C。对于完全二叉树，第 6 层有叶节点，说明树要么 6 层，要么 7 层。显然树为 7 层时节点更多。因此要达到节点最多，树形一定如下图：



所以最大节点数就是，前六层是满的：1 + 2 + 4 + 8 + 16 + 32 = 63，再加上第七层的节点数 48，共 111。

通过本题，也给了我们一个提示：**高为 h 的二叉树，节点数最大是** $1+2+...+2^{h-1}=2^h-1$。该性质同样可以作为二叉树的性质记忆，也可以根据每层最大节点数用等比数列求和公式现推。

12. 选 C。最直观（但复杂）的想法应该就是先算出二叉树有多少层、最后一层有多少节点，这样就能求出倒数第二层的叶节点的数量，再加上最后一层的节点数就是总共的叶节点数。

更简单的做法是，利用**本习题第 8 小题给出的性质 c）**：最后一个非叶节点的编号是 $\lfloor \frac{768}{2} \rfloor = 384$，即非叶节点有 384 个。所以叶节点数量为 768 - 384 = 384 个。

还有一种不太直观的解法，利用**本习题第 5 小题给出的性质）**：$n_0 = n_2+1$，有 $2n_0+n_1-1=768$，又因为完全二叉树中 n_1 只能取 1 或 0，当 n_1 取 0 时，n_0 为小数，显然不行，所以 n_1 只能取 1，得到 $n_0 = 384$。

13. 选 A。要满足任意一棵高度为 5 且有 10 个节点的二叉树都能存得下，需要考虑最坏情况，即存储单元消耗最大的情况。顺序存储通过从上到下，从左到右依次保存每一个位置的节点，高度为 5，则最坏情况是第五层最右边的节点被占用，需要 31 个存储空间。而 10 个节点能否达到此情况？容易得知是可以的。所以选 A，31。

14. 选 B。树的度为 4，说明树中只有度为 0、1、2、3、4 的节点，又已知了度为 1、2、3、4 的节点数量分别为 10、1、10、20，根据“树的所有节点度数之和等于树的边数”的性质，树的边数可以直接求得：

$$e = 10*1+1*2+10*3+20*4 = 122$$

于是树的节点数 $n = 122+1 = 123$。于是叶节点，即度为 0 的节点数为 $123-10-1-10-20 = 82$。

15. 选 C。虽然本题目提出的存储方式与 1.2.1 小节中提出的二叉树的顺序存储物理上都是使用一个数组存储，但逻辑上对于二叉树中节点关系的描述方式不同。A、B、D 显然不合逻辑。对于 C

项，数组的长度是确定的，若某场景下无法确定需要使用的二叉树是否有节点数上限，则不适合使用固定大小的数组存储，C 正确。

16. 第 1）小问很简单，将树遍历一遍就行。这里拟用先序遍历，代码如下：

```
void travel(node* root) {
    if(root == NULL) return ;
    if(root->data == 5) root->data = 10;
    travel(root->lchild);
    travel(root->rchild);
}
void five2ten(node* T) {
    travel(T);
}
```

事实上，第 2）问也不难。定义一个 int 变量，这里取名为 ret，标识是否有元素的值被修改过。递归时将 ret 的地址传入递归函数，在某节点处修改了值时，将该 ret 的值置为 1 就好。代码如下：

```
void travel(node* root, int* value_changed) {
    if(root == NULL) return ;
    if(root->data == 5) {
        root->data = 10;
        *value_changed = 1;
    }
    travel(root->lchild, value_changed);
    travel(root->rchild, value_changed);
}
int five2ten(node* T) {
    int ret = 0;    // 此时未修改任何值
    travel(T, &ret);
    return ret;
}
```

还是提示一下，value_changed 只是形参，它实际传入的实参是 five2ten() 中 ret 这个变量的地址。

17. 1）代码如下：

```
void PostOrderTravel(node* root) {
    if(root == NULL) return ;
    PostOrderTravel(root->lchild);
    PostOrderTravel(root->rchild);
    printf("%c", root->data);
}
void f(node* T) {
    PostOrderTravel(T);
    printf("\n");
}
```

2）后序遍历序列为 CoreLoop。

18. 1）通过观察不难发现，对于树中的任意一个节点，其左子树中所有节点的值均小于或等于

该节点的值，右子树中所有节点的值均大于该节点的值。
2）中序遍历，得到的序列是 1, 4, 4, 5, 6, 7, 8, 9, 10, 10；该序列是大小有序的。
3）插入节点时，可以从根节点开始，将根节点元素与待插元素比较。若待插元素较小（或等于），则说明该节点需要在左子树中插入，此时，若：
① 左子树为空，则插入位置已经找到，直接插入；
② 左子树非空，则递归地进入左子树，进行同样的操作。
反之若待插元素大于根节点，则递归地判断右子树。如此递归进行下去，直至找到正确的位置。具体实现代码如下：

```
void InsertTree(node* &root, int e) {
    if(root == NULL) {
        // 空树，直接插入就行
        node* new_node = (node*)malloc(sizeof(node));
        new_node->data = e;
        new_node->lchild = NULL;
        new_node->rchild = NULL;
        root = new_node;
        return ;
    }
    if(e <= root->data) {   // 待插元素较小
        if(root->lchild == NULL) {  // 左子树为空，说明找到了插入位置
            node* new_node = (node*)malloc(sizeof(node));
            new_node->data = e;
            new_node->lchild = NULL;
            new_node->rchild = NULL;
            root->lchild = new_node;
        } else {    // 递归地进入左子树
            InsertTree(root->lchild, e);
        }
    }
    if(e > root->data) {    // 待插元素较大
        if(root->rchild == NULL) {  // 右子树为空，说明找到了插入位置
            node* new_node = (node*)malloc(sizeof(node));
            new_node->data = e;
            new_node->lchild = NULL;
            new_node->rchild = NULL;
            root->rchild = new_node;
        } else {   // 递归地进入右子树
            InsertTree(root->rchild, e);
        }
    }
}
```

事实上，我们将在后面的内容中介绍到，有着这样特点的树叫做**二叉查找树**，或者叫**二叉搜索树**、**二叉排序树**。正如其名，这种数据结构可以利用节点值之间的大小关系进行较为高效的查找操作，我们将在后面的内容中详细介绍。

—— BONFIRE HIT ——

谢谢你带我来到这里，伴火同进者，终有一天会遇见命定之死，永别了。